STL_plus

David Schwarz

2025-09-10

Read generated Data

Some modifications to the dataset

```
# remove the last 54322 rows to not include the gap in the data
n <- nrow(data)
data <- data[1:(n - 54322), ]

# Convert time to POSIXct (datetime - should already be correct but just to be sure)
data$time <- as.POSIXct(data$time, format="%Y-%m-%d %H:%M:%S")
```

split dataset into subgroups

```
# Example split dates
split_dates <- c("2023-05-01 00:00:00",
                 "2023-10-01 00:00:00",
                 "2024-03-01 00:00:00")

# Convert to POSIXct
split_dates <- as.POSIXct(split_dates, format="%Y-%m-%d %H:%M:%S")

# Define interval boundaries
breaks <- c(min(data$time), split_dates, max(data$time))

# Create nice labels for each interval
labels <- c(
  paste0(format(breaks[1], "%Y-%m-%d"), "_to_", format(breaks[2], "%Y-%m-%d")),
  paste0(format(breaks[2], "%Y-%m-%d"), "_to_", format(breaks[3], "%Y-%m-%d")),
  paste0(format(breaks[3], "%Y-%m-%d"), "_to_", format(breaks[4], "%Y-%m-%d")),
  paste0(format(breaks[4], "%Y-%m-%d"), "_to_", format(breaks[5], "%Y-%m-%d"))
)

# Assign groups
data$group <- cut(
  data$time,
  breaks = breaks,
  include.lowest = TRUE,
  right = FALSE, # ensures [start, end)
  labels = labels
)

# Split dataset by group
data_groups <- split(data, data$group)
```

```r
# Preview sizes
sapply(data_groups, nrow)
```

```
## 2023-01-02_to_2023-05-01 2023-05-01_to_2023-10-01 2023-10-01_to_2024-03-01
##                   341040                   440640                   437880
## 2024-03-01_to_2024-05-13
##                   212197
```

STL Function

```r
library(stlplus)
```

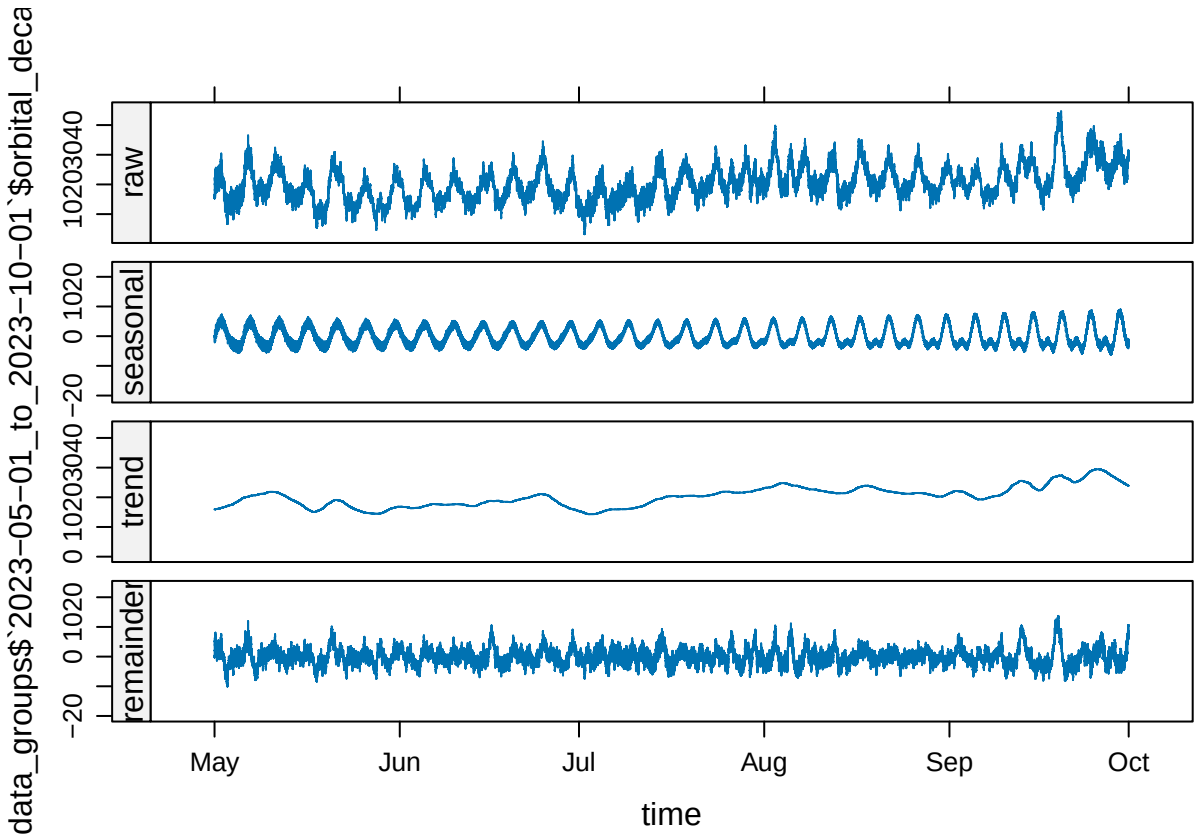
```
## Warning: package 'stlplus' was built under R version 4.4.3
```

```r
stl_decompose <- function(period_days, x, time) {
  # Benchmark start
  start_time <- Sys.time()

  # Convert days to half minutes (data sample rate)
  period <- period_days * 24 * 120

  # Make sure seasonal period is odd
  seasonal <- ifelse(period %% 2 == 0, period + 1, period)

  # Jumps / windows
  low_pass_window <- max(101, round(period))          # ~period length
  trend_window    <- max(301, round(1.5 * period))    # longer, smooth trend

  s_jump <- floor(0.15 * (period + 1))
  l_jump <- s_jump
  t_jump <- floor(0.15 * 1.5 * (period + 1))

  stl_result <- stlplus(
    x = x,
    t = time,
    n.p = period,
    s.window = seasonal,      # keeps seasonal period
    l.window = low_pass_window,
    t.window = trend_window,
    s.jump   = s_jump,
    l.jump   = l_jump,
    t.jump   = t_jump
  )

  # Plot result
  print(
    plot(
      stl_result,
      xlab = "time",
      ylab = deparse(substitute(x))
    )
  )
```

```
  # Benchmark end
  end_time <- Sys.time()
  cat("Runtime:", round(difftime(end_time, start_time, units = "secs"), 2), "seconds\n")
  # statistics
  cat("Using a period of", period_days, "days →", period, "points\n")
  cat("Datapoints: ", length(x), "\n")

  # Extract components
  comp <- data.frame(
    seasonal = stl_result$data$seasonal,
    trend    = stl_result$data$trend,
    remainder= stl_result$data$remainder
  )

  # Compute statistics
  stats <- data.frame(
    component = c("seasonal", "trend", "remainder"),
    mean      = c(mean(comp$seasonal), mean(comp$trend), mean(comp$remainder)),
    sd        = c(sd(comp$seasonal),   sd(comp$trend),   sd(comp$remainder)),
    min       = c(min(comp$seasonal),  min(comp$trend),  min(comp$remainder)),
    max       = c(max(comp$seasonal),  max(comp$trend),  max(comp$remainder))
  )

  print(stats)

  invisible(stl_result)
}
```

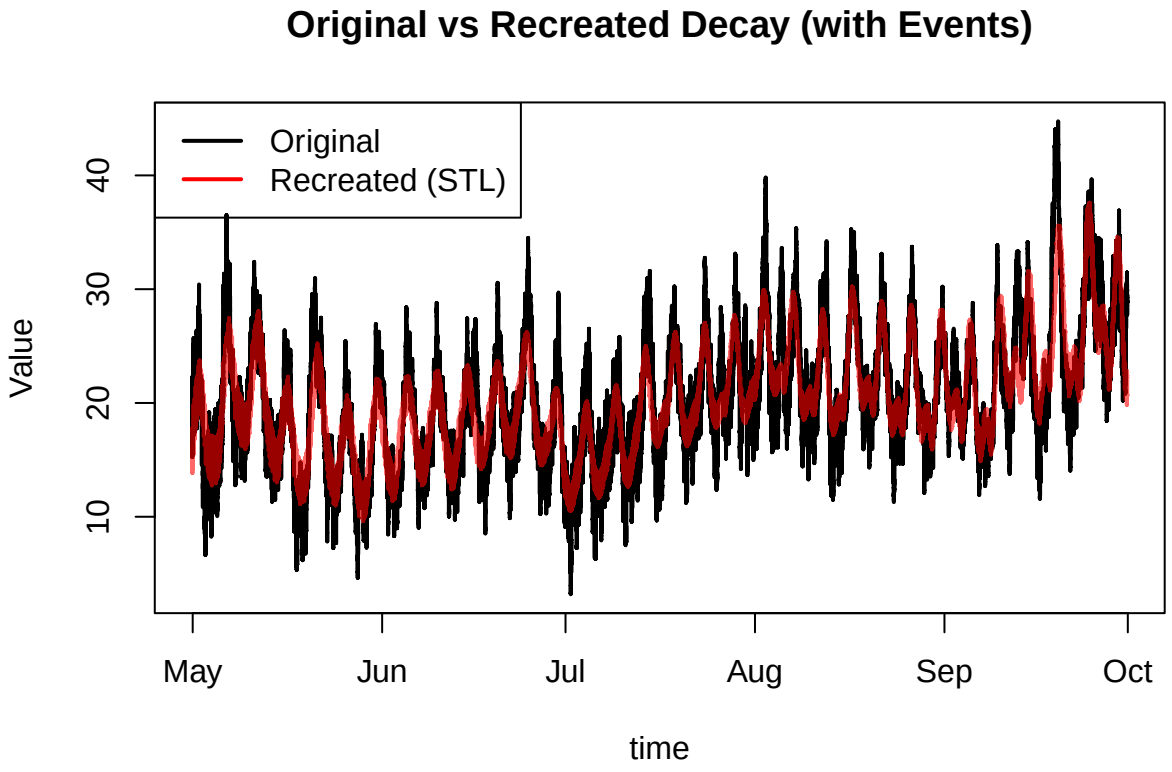
STL Decay of 2. subgroup

```
res_D <- stl_decompose(
  period_days = 4.82,
  x = data_groups$`2023-05-01_to_2023-10-01`$orbital_decay,

  time = data_groups$`2023-05-01_to_2023-10-01`$time
)
```




```
## Runtime: 600.9 seconds
## Using a period of 4.82 days → 13881.6 points
## Datapoints:  440640
##   component         mean       sd       min       max
## 1  seasonal  0.044233716 3.259250  -6.29153  8.989239
## 2     trend 20.255225418 3.381888  14.29898 29.474075
## 3 remainder  0.009953229 2.894973 -10.09190 13.724911
```

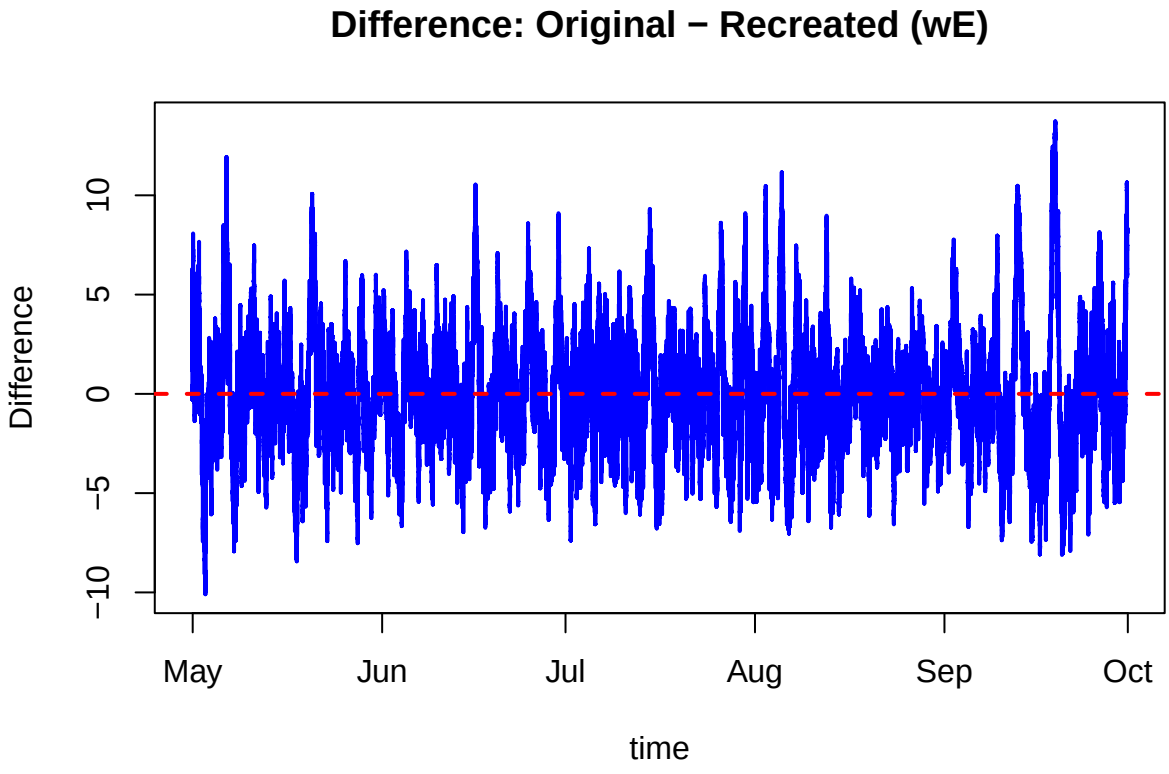
```r
# Recreated (trend + seasonal)
recreated <- res_D$data$seasonal + res_D$data$trend

orig <- data_groups$`2023-05-01_to_2023-10-01`$orbital_decay

# Compare visually
plot(data_groups$`2023-05-01_to_2023-10-01`$time, orig, type = "l", col = "black", lwd = 2,
     main = "Original vs Recreated Decay (with Events)",
     xlab = "time", ylab = "Value")
lines(data_groups$`2023-05-01_to_2023-10-01`$time, recreated, col = adjustcolor("red", alpha.f = 0.6),
legend("topleft", legend = c("Original", "Recreated (STL)"),
       col = c("black", "red"), lty = 1, lwd = 2)
```




```
# Compare numerically (ignoring NAs)
mean_diff <- mean(abs(orig - recreated), na.rm = TRUE)
cat("Mean absolute difference:", mean_diff, "\n")
```

```
## Mean absolute difference: 2.230029
```

```
# Difference (residuals)
diff <- orig - recreated

# Plot the difference over time
plot(data_groups$`2023-05-01_to_2023-10-01`$time, diff, type = "l", col = "blue", lwd = 2,
     main = "Difference: Original - Recreated (wE)",
     xlab = "time", ylab = "Difference")

abline(h = 0, col = "red", lty = 2, lwd = 2)  # reference line at 0
```



Test

```
comp <- data.frame(
  seasonal = res_D$data$seasonal,
  trend    = res_D$data$trend,
  remainder= res_D$data$remainder
)

# Compute statistics
stats <- data.frame(
  component = c("seasonal", "trend", "remainder"),
  mean      = c(mean(comp$seasonal), mean(comp$trend), mean(comp$remainder)),
  sd        = c(sd(comp$seasonal),   sd(comp$trend),   sd(comp$remainder)),
  min       = c(min(comp$seasonal),  min(comp$trend),  min(comp$remainder)),
  max       = c(max(comp$seasonal),  max(comp$trend),  max(comp$remainder))
)

print(stats)
```

```
##   component         mean       sd       min       max
## 1  seasonal  0.044233716 3.259250  -6.29153  8.989239
## 2     trend 20.255225418 3.381888  14.29898 29.474075
## 3 remainder  0.009953229 2.894973 -10.09190 13.724911
```

STL Decay with Gaps where Events are

Import filtered data

```
# Path to the CSV file
filtered_path <- path(repo_root, "Dataset", "Dataset_MSc", "filtered_GFOC_data.csv")
# Try to load with limited columns first
fdata <- tryCatch(
  {
    # Use readr::read_csv for efficiency (similar to pandas)
    dat <- read_csv(filtered_path, col_select = all_of(columns_needed))
    message(sprintf("Successfully loaded data with shape: %d rows, %d cols",
                    nrow(dat), ncol(dat)))
    dat
  },
  error = function(e) {
    if (grepl("cannot allocate memory", e$message)) {
      message("Still running out of memory. Loading in chunks...")

      # Chunked read
      chunk_list <- list()
      chunksize <- 50000

      read_csv_chunked(
        GFOC_path,
        callback = DataFrameCallback$new(function(x, pos) {
          chunk_list[[length(chunk_list) + 1]] <<- x
        }),
        col_select = all_of(columns_needed),
        chunk_size = chunksize
      )

      dat <- do.call(rbind, chunk_list)
      message(sprintf("Successfully loaded data in chunks with shape: %d rows, %d cols",
                      nrow(dat), ncol(dat)))
      dat
    } else {
      stop(e)
    }
  }
)
```

```
## Rows: 1486079 Columns: 2
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## dbl  (1): orbital_decay
## dttm (1): time
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
## Successfully loaded data with shape: 1486079 rows, 2 cols
```

Reformat

```
# remove the last 54322 rows to not include the gap in the data
n <- nrow(fdata)
fdata <- fdata[1:(n - 54322), ]

# Convert time to POSIXct (datetime - should already be correct but just to be sure)
fdata$time <- as.POSIXct(data$time, format="%Y-%m-%d %H:%M:%S")
```

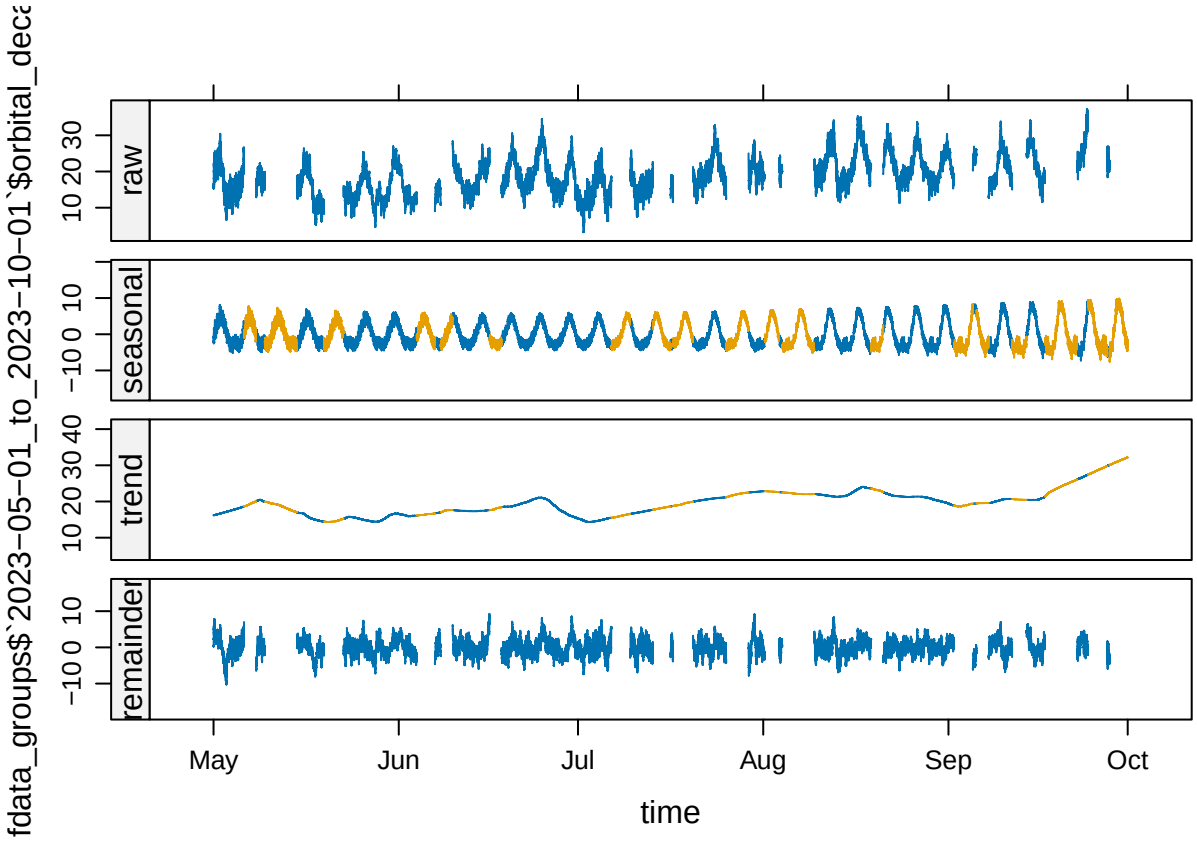
split dataset into subgroups

```
# Assign groups
fdata$group <- cut(
  fdata$time,
  breaks = breaks,
  include.lowest = TRUE,
  right = FALSE, # ensures [start, end)
  labels = labels
)

# Split dataset by group
fdata_groups <- split(fdata, fdata$group)

# Preview sizes
sapply(fdata_groups, nrow)
```

```
## 2023-01-02_to_2023-05-01 2023-05-01_to_2023-10-01 2023-10-01_to_2024-03-01
##                   341040                   440640                   437880
## 2024-03-01_to_2024-05-13
##                   212197
```

STL

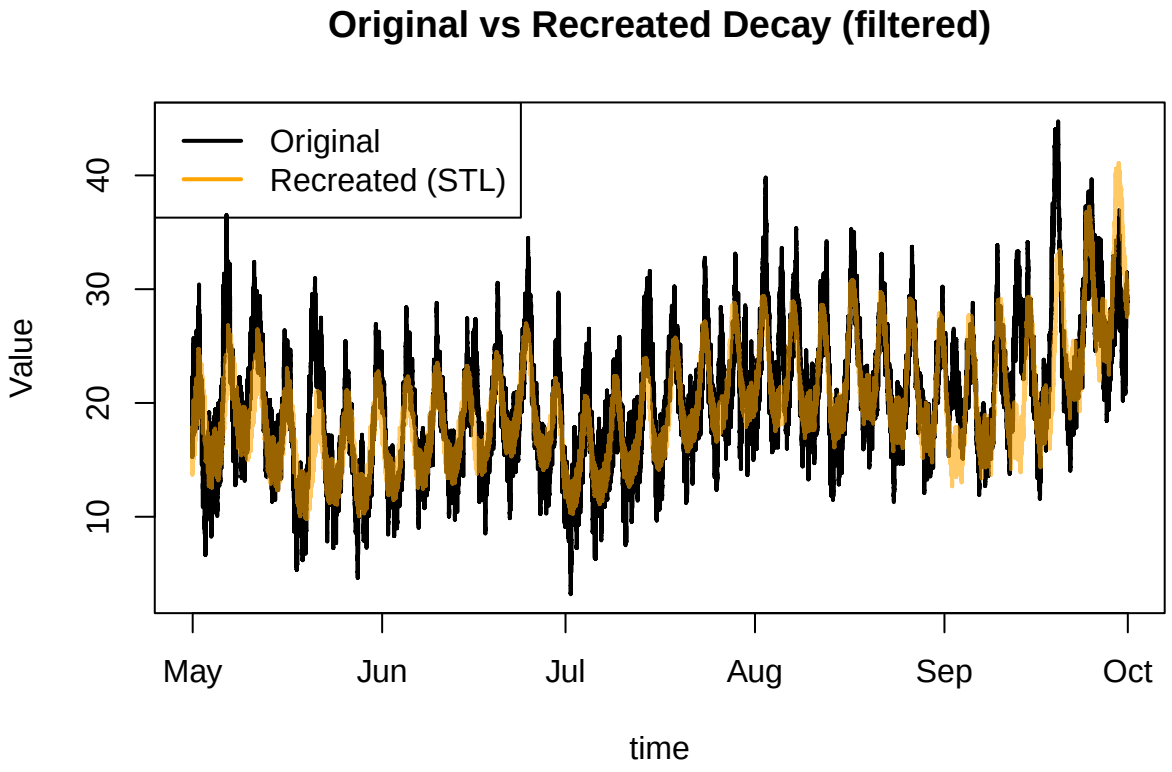
```
res_fD <- stl_decompose(
  period_days = 4.82,
  x = fdata_groups$`2023-05-01_to_2023-10-01`$orbital_decay,

  time = fdata_groups$`2023-05-01_to_2023-10-01`$time
)
```




```
## Runtime: 401.2 seconds
## Using a period of 4.82 days → 13881.6 points
## Datapoints:  440640
##   component        mean       sd       min       max
## 1  seasonal  0.04603058 3.507279 -7.604618  9.848729
## 2     trend 19.65887392 3.529616 14.314718 32.197957
## 3 remainder          NA       NA        NA        NA
```

Recreate Decay without Events

```r
# Recreated (trend + seasonal)
recreated <- res_fD$data$seasonal + res_fD$data$trend

orig <- data_groups$`2023-05-01_to_2023-10-01`$orbital_decay

# Compare visually
plot(data_groups$`2023-05-01_to_2023-10-01`$time, orig, type = "l", col = "black", lwd = 2,
     main = "Original vs Recreated Decay (filtered)",
     xlab = "time", ylab = "Value")
lines(data_groups$`2023-05-01_to_2023-10-01`$time, recreated, col = adjustcolor("orange", alpha.f = 0.6
legend("topleft", legend = c("Original", "Recreated (STL)"),
       col = c("black", "orange"), lty = 1, lwd = 2)
```

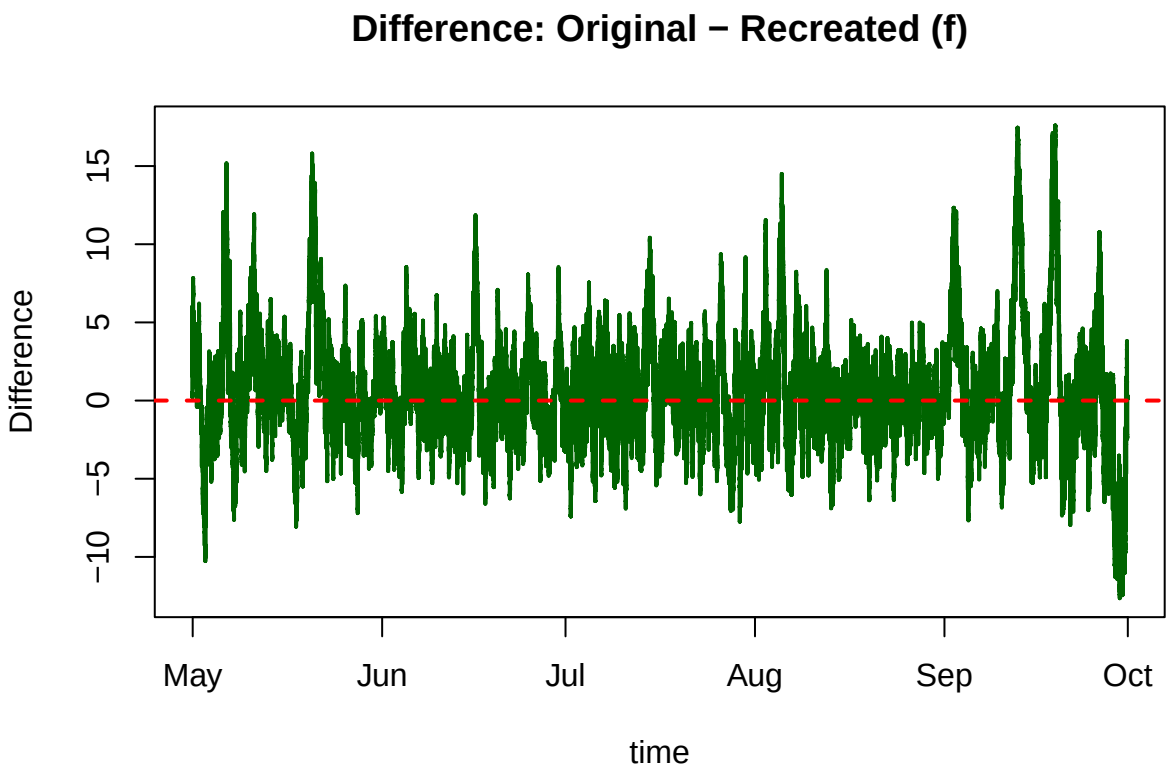



```
# Compare numerically (ignoring NAs)
mean_diff <- mean(abs(orig - recreated), na.rm = TRUE)
cat("Mean absolute difference:", mean_diff, "\n")
```

```
## Mean absolute difference: 2.548688
```

Difference between Original and Recreated

```
# Difference (residuals)
diff <- orig - recreated

# Plot the difference over time
plot(data_groups$`2023-05-01_to_2023-10-01`$time, diff, type = "l", col = "darkgreen", lwd = 2,
     main = "Difference: Original - Recreated (f)",
     xlab = "time", ylab = "Difference")

abline(h = 0, col = "red", lty = 2, lwd = 2)  # reference line at 0
```



create csv

```
# Add the vectors as new columns to the dataframe
data_groups$`2023-05-01_to_2023-10-01`$recreated  <- recreated
data_groups$`2023-05-01_to_2023-10-01`$difference <- diff

# Save the updated dataframe as CSV
write.csv(
  data_groups$`2023-05-01_to_2023-10-01`,
  file = "STLplus_2023-05-01_to_2023-10-01.csv",
  row.names = FALSE
)
```

Lomb-Scargle

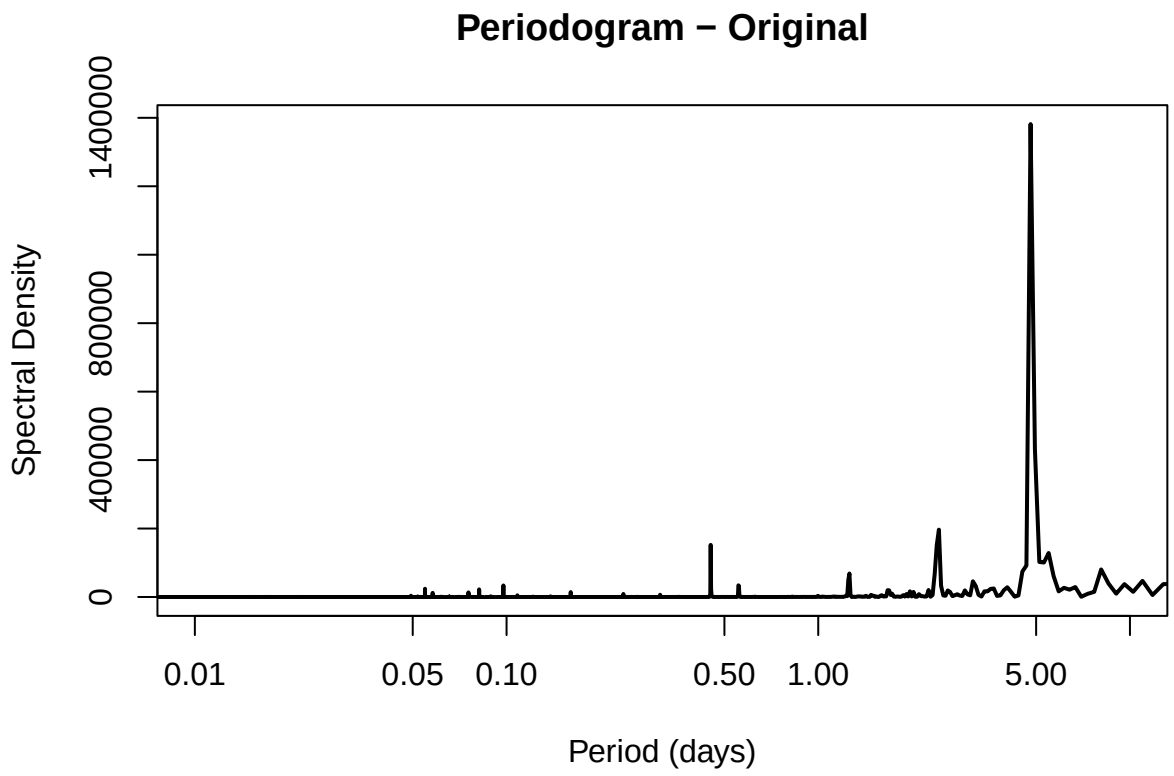
```
library(lubridate)
```

```
##
## Attaching package: 'lubridate'
```

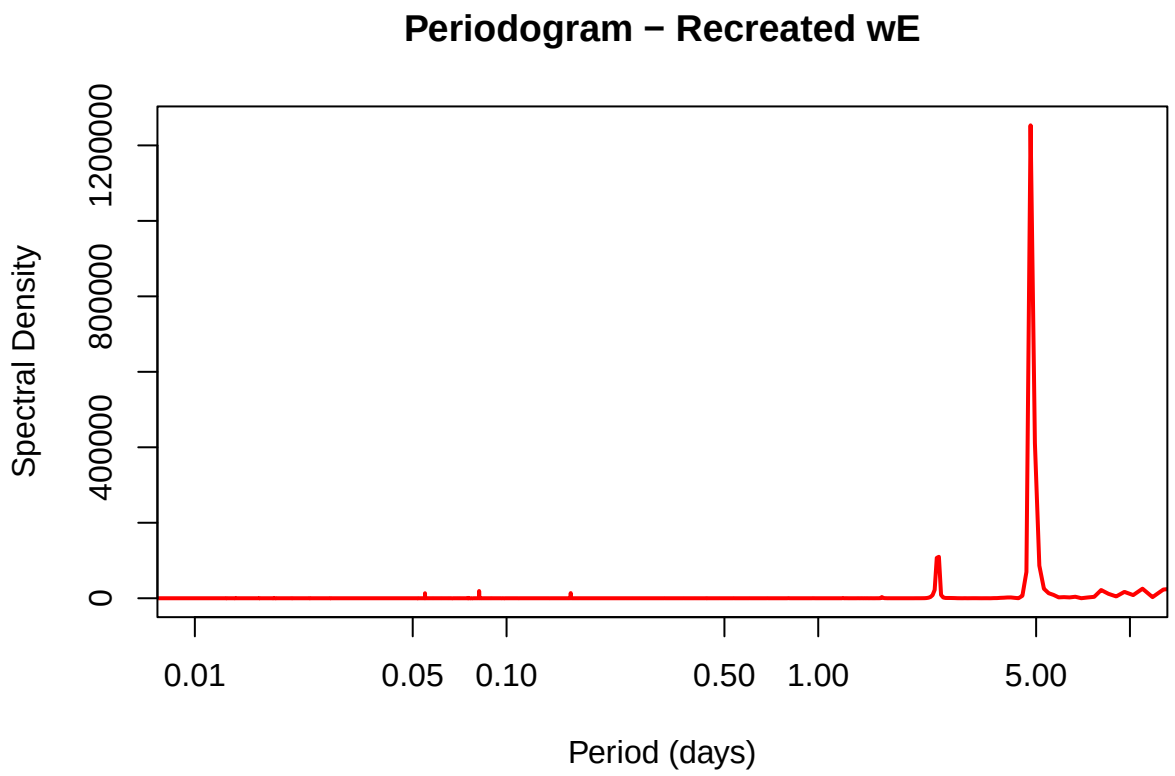
```
## The following objects are masked from 'package:base':
##
##     date, intersect, setdiff, union
```

```
df <- data_groups$`2023-05-01_to_2023-10-01`

# Ensure time is POSIXct
df$time <- as.POSIXct(df$time, format="%Y-%m-%d %H:%M:%S")
```

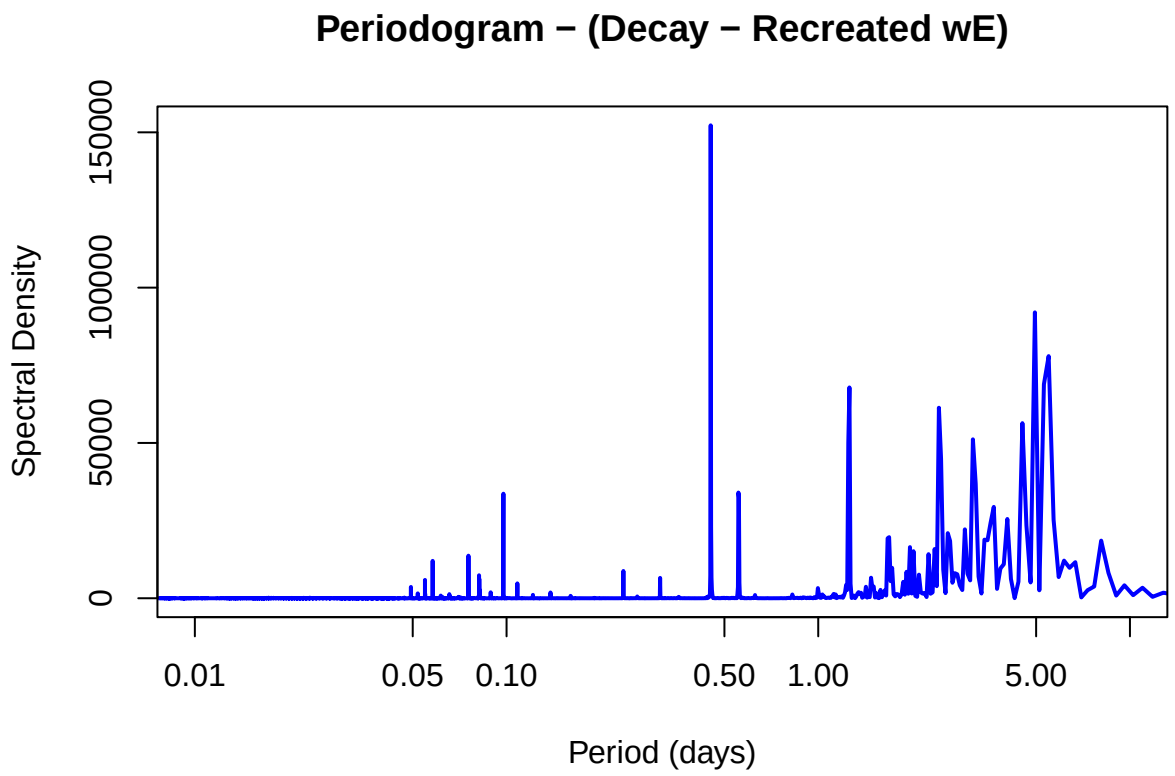
```r
# Sampling interval in days
dt <- as.numeric(difftime(df$time[2], df$time[1], units = "days"))

# --- Periodogram for 'decay' ---
spec_decay <- spectrum(df$orbital_decay, plot = FALSE)
periods_decay <- 1 / (spec_decay$freq / dt)

plot(periods_decay, spec_decay$spec, type = "l",
     col = "black", lwd = 2,
     log = "x",
     xlab = "Period (days)", ylab = "Spectral Density",
     main = "Periodogram - Original",
     xlim = c(1e-2, 10)
)
```



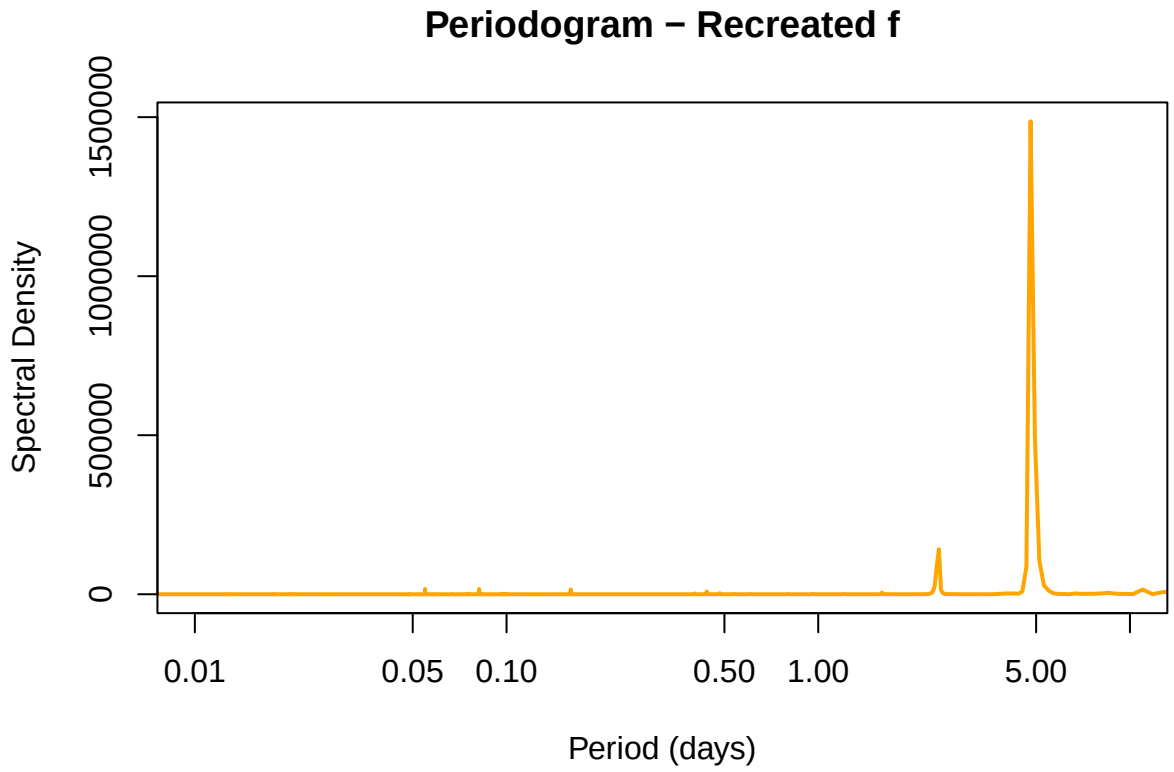

```r
# --- Periodogram for 'RwE' ---
spec_STL <- spectrum(res_D$data$seasonal + res_D$data$trend, plot = FALSE)
periods_STL <- 1 / (spec_STL$freq / dt)

plot(periods_STL, spec_STL$spec, type = "l",
     col = "red", lwd = 2,
     log = "x",
     xlab = "Period (days)", ylab = "Spectral Density",
     main = "Periodogram - Recreated wE",
     xlim = c(1e-2, 10)
)
```



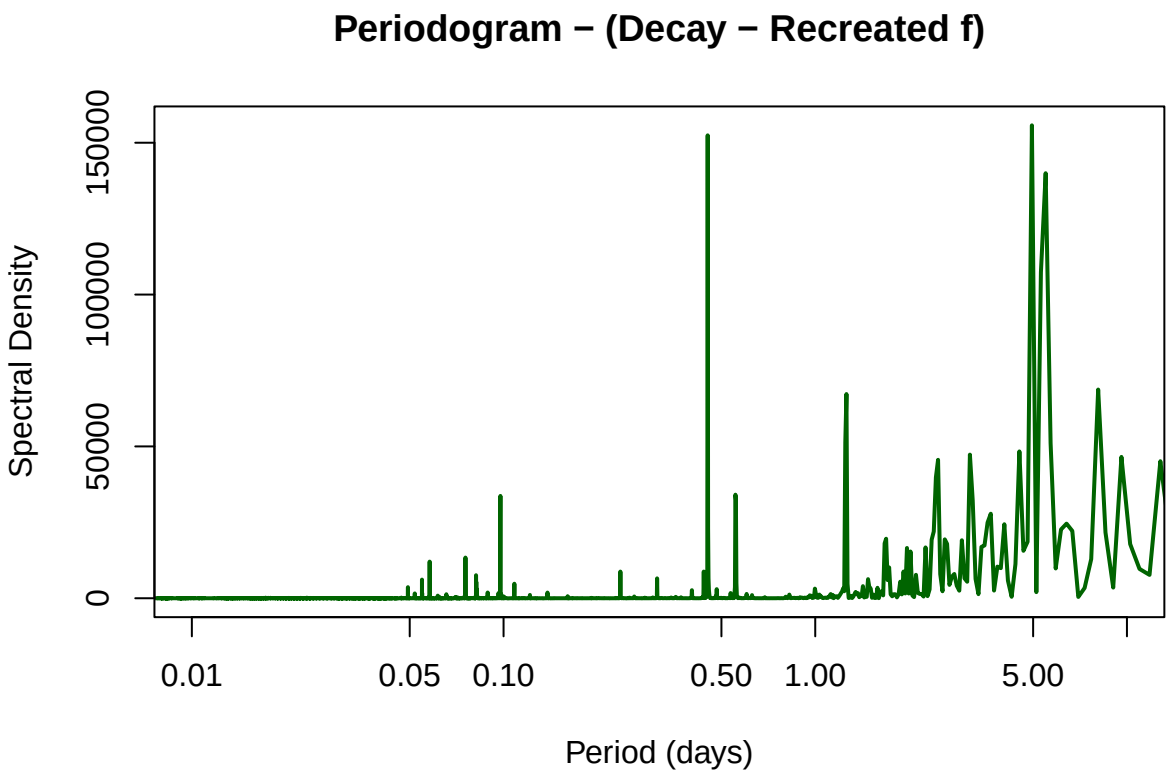

```r
# --- Periodogram for 'Decay - RwE' ---
spec_DSTL <- spectrum(df$orbital_decay - (res_D$data$seasonal + res_D$data$trend), plot = FALSE)
periods_DSTL <- 1 / (spec_DSTL$freq / dt)

plot(periods_DSTL, spec_DSTL$spec, type = "l",
     col = "blue", lwd = 2,
     log = "x",
     xlab = "Period (days)", ylab = "Spectral Density",
     main = "Periodogram - (Decay - Recreated wE)",
     xlim = c(1e-2, 10)
)
```




```r
# --- Periodogram for 'recreated f' ---
spec_recreated <- spectrum(df$recreated, plot = FALSE)
periods_recreated <- 1 / (spec_recreated$freq / dt)

plot(periods_recreated, spec_recreated$spec, type = "l",
     col = "orange", lwd = 2,
     log = "x",
     xlab = "Period (days)", ylab = "Spectral Density",
     main = "Periodogram - Recreated f",
     xlim = c(1e-2, 10)
)
```

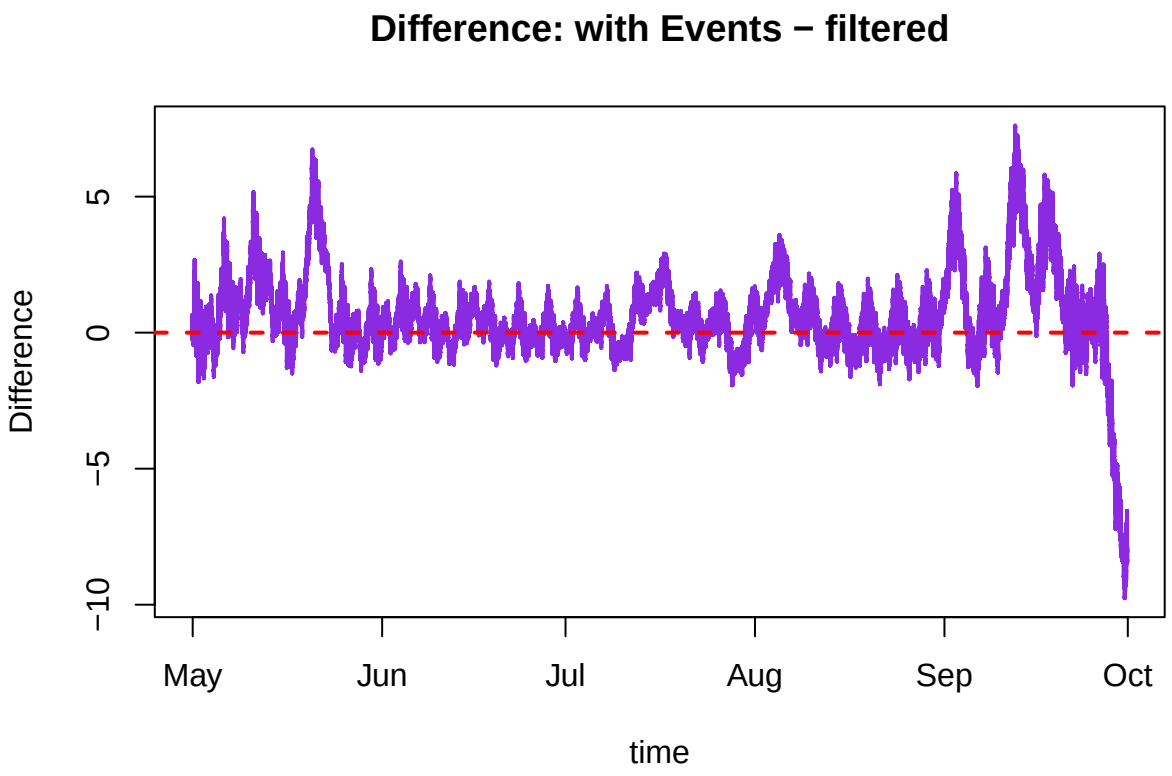



```r
# --- Periodogram for 'Decay - Rf' ---
spec_diff <- spectrum(df$difference, plot = FALSE)
periods_diff <- 1 / (spec_diff$freq / dt)

plot(periods_diff, spec_diff$spec, type = "l",
     col = "darkgreen", lwd = 2,
     log = "x",
     xlab = "Period (days)", ylab = "Spectral Density",
     main = "Periodogram - (Decay - Recreated f)",
     xlim = c(1e-2, 10)
)
```



Difference between STL with Gaps and STL with original data

```r
# Recreated curve
full <- res_D$data$seasonal + res_D$data$trend
gaps <- res_fD$data$seasonal + res_fD$data$trend

df <- full - gaps

# Plot the difference over time
plot(data_groups$`2023-05-01_to_2023-10-01`$time, df, type = "l", col = "blueviolet", lwd = 2,
     main = "Difference: with Events - filtered",
     xlab = "time", ylab = "Difference")

abline(h = 0, col = "red", lty = 2, lwd = 2)  # reference line at 0
```


Difference: with Events − filtered
Difference
5
0
−5
−10
May
Jun
Jul
Aug
Sep
Oct
time